FUNDACIÓN UNIVERSITARIA AGRARIA DE COLOMBIA – UNIAGRARIA
Dirección de Educación Virtual · Ciencia de Datos (Profesional)
Actividad 2 – Hands on: configuración de herramientas
Estudiante: Andres Felipe Amaya Camacho
Docente: Carlos Andres Gomez Vasco

Capítulo 2. Fundamentos de Git

Crear repositorios, registrar cambios, consultar el historial, deshacer y trabajar con remotos

Este capítulo recoge los comandos que se usan a diario: crear o clonar un repositorio, registrar cambios, revisar el historial, corregir errores, sincronizar con un remoto y etiquetar versiones.

2.1 Obtener un repositorio

Hay dos caminos: convertir una carpeta existente en repositorio con `git init`, o copiar uno existente con `git clone`, que descarga toda la historia.

```
git init
git clone https://github.com/usuario/repositorio.git
git clone https://github.com/usuario/repositorio.git mi-carpeta
```

2.2 Guardar cambios en el repositorio

Cada archivo está *rastreado* (tracked) o *sin rastrear* (untracked). El ciclo de trabajo es: modificar, preparar con `git add` y confirmar con `git commit`.

```
git status                # estado de los archivos
git status -s             # formato corto
git add archivo.txt       # prepara un archivo
git add .                 # prepara todo
git diff                  # cambios sin preparar
git diff --staged         # cambios preparados
git commit -m "Mensaje claro y descriptivo"
git commit -a -m "Mensaje"  # omite git add para archivos ya rastreados
```

Para eliminar o mover archivos se usan `git rm` y `git mv`, que además dejan el cambio preparado. Los patrones de archivos que no deben rastrearse (datos temporales, credenciales, entornos virtuales) se listan en `.gitignore`.

```
git rm archivo.txt
git mv origen.txt destino.txt
```

2.3 Ver el historial de confirmaciones

`git log` muestra los commits del más reciente al más antiguo. Sus opciones permiten adaptarlo a lo que se busca.

```
git log
git log --oneline
git log --oneline --graph --decorate
git log -p -2          # últimos 2 commits con sus diferencias
git log --stat
git log --since="2 weeks" --author="Andres"
```

2.4 Deshacer cosas

Algunas operaciones son reversibles y otras no, así que conviene saber cuál se está usando:

- **Rehacer el último commit** (por olvidar un archivo o corregir el mensaje): `git commit --amend`.
- **Sacar un archivo del área de preparación:** `git restore --staged archivo` (antes `git reset HEAD archivo`).
- **Descartar cambios locales de un archivo:** `git restore archivo` (antes `git checkout -- archivo`). Esta acción *no se puede deshacer*.

2.5 Trabajar con remotos

Un remoto es una versión del proyecto alojada en otro lugar, como GitHub. Al clonar, Git crea automáticamente el remoto `origin`.

```
git remote -v
git remote add origen https://github.com/usuario/repo.git
git fetch origin         # descarga sin fusionar
git pull                 # fetch + merge
git push origin main     # envía los commits locales
git remote show origin
git remote rename origen origin
git remote remove origen
```

2.6 Etiquetas

Las etiquetas (tags) marcan puntos importantes de la historia, normalmente versiones (v1.0). Pueden ser ligeras (solo un puntero) o anotadas (con autor, fecha y mensaje, recomendadas).

```
git tag
git tag -a v1.0 -m "Versión 1.0"
git show v1.0
git push origin v1.0
git push origin --tags
```

2.7 Alias de Git

Los alias acortan comandos largos:

```
git config --global alias.co checkout
git config --global alias.st status
git config --global alias.lg "log --oneline --graph --decorate --all"
```

Conclusión del capítulo

Estos comandos forman el flujo básico: clone → modificar → add → commit → push. Dominarlos y escribir buenos mensajes de confirmación es la base de cualquier proyecto con Git.

Fuente: Chacon, S. y Straub, B. (2014). Pro Git (2.ª ed.). Apress. https://git-scm.com/book/es/v2